

順序を見つける

- 制限時間: 10 分
- 環境: GPU 1 基 (≈16 GB VRAM) 、インターネット接続なし
- 解答サイズ: `solution.ipynb` ≤ 1 MB
- ストレージ: 5 GB

問題

2 人の参加者、**Speaker A** と **Speaker B** による英語の音声□話が与えられます。各□話は話者のタ□ンに分割されており、各タ□ンには一方の話者の□□のみが含まれます。各タ□ンは個□の .wav 音声ファイルとして保存されているため、完全な□□は、各タ□ンあたり1つの .wav ファイルからなる集合として表されます。

残念ながら、タ□ンはランダムにシャッフルされているため、会話はもはや意味をなしません。ファイル名 `chunk_{k}.wav` において、 k は元の□□における k 番目のタ□ンではなく、シャッフルされた集合における k 番目のチャンクを指します。

!! あなたの課題は、会話の元の時系列順を復元することです。



データセット

各話には、`chunk_0.wav`、`chunk_1.wav`、...、`chunk_{n-1}.wav` という名前の n 音声ファイルが含まれています。チャンクは個々のターンです。ファイル名はシャッフル後の順序に並んでいます。ファイル名は、チャンクが元の会話のどこに属するかを示すものではありません。各話には 7–20 個のチャンクがあり、モノラル、44.1 kHz です（リサンプリングしてもかまいません）。

prefix.json には、各話の最初の 2 つのチャンクのファイル名インデックスが含まれています。これにより、真の開始部分が特定され、会話を前向きに読むか後ろ向きに読むかという違いがなくなります。

例：11: [7, 12] は、話 11 の 1 番目と 2 番目のターンが、それぞれ `chunk_7.wav` と `chunk_12.wav` であることを意味します。

提供されるもの

同一形式の 2 つのフォルダが提供されます。

フォルダ	行数	answers.json?	用途
<code>dataset/train/</code>	1,288	含まれる	モデルの訓練 / fine-tuning
<code>dataset/test_public/</code>	100	含まれる	パイプラインを評価し、ロカルで自己評価

本データセットには、`dataset/test_public/` フォルダが hidden evaluation set（公開リデータセット用の `test_leaderboard_a` と最終リデータセット用の `test_leaderboard_b`）に置き換えられます。これらは `dataset/test_public/` と同じサイズおよび形式ですが、`answers.json` は含まれていません。

そのデータ上で **notebook** が再度実行され、生成された `answers.json` ファイルが本データセットに使用されます。ホールドアウトされたテストデータは `train` と同じ分布に由来するため、ロカルの `test_public` スコアは忠実な事前評価となります。

ディレクトリ構造

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

出力

各話について、音声チャンクの元の時系列順を求めてください。予測は $\{0, 1, \dots, n-1\}$ の置換 P でなければなりません。ここで、 $P[i]$ は `chunk_i.wav` の予測された時系列上の位置です ($0 =$ 最初)。

出力ファイル `answers.json` では、各話 ID をその予測された置換にマッピングしてください。

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

例

ある話には、シャッフルされた **3** 個のチャンク `chunk_0`, `chunk_1`, `chunk_2` があります。

シャッフルされたチャンク	□□□□	真の位置（順位）
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2（最後）
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0（最初）
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

真の順序は **chunk_1** → **chunk_2** → **chunk_0** であるため、 $P = [2, 0, 1]$ となり、`prefix.json` には $[1, 2]$ が格納されます。

△ **P は正しい置□されたリストでなければなりません**: 長さ n 、0 始まりで、各□がちょうど 1 回ずつ現れる必要があります。重□、欠損□、範□外の要素（例: 1 始まり）がある場合、その□□のスコアは 0 になります。ファイルに含まれていない□□についても同□です。形式が不正なファイル、または JSON でないファイルは拒否されます。

□□

この課題の□□□□は**ペアごとの順序正解率**です。すべてのチャンクのペアについて、どちらが先に来るべきかを確認します。予測が正解と同じ答えを与えた場合、そのペアは正解です。 n 個のチャンクを持つ□□には

$$M = n(n - 1)/2$$

個のペアがあります。 I を逆□数、すなわち正解とは□なる順序に□べられたペアの数とします。

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i 最終スコアは、その分割内のすべての□□について、□□ごとのスコアを平均したものです。

使用可能なモデル

訓練時と評価時の□□において、この課題を解くために使用できる事前訓練□みモデルは以下のものだけです。これらのモデルはすべて、すでにダウンロードされ、環境内で利用可能です。ペ□スライン `notebook solution.ipynb` で使用例を確認できます。他のモデルは使用できず、プログラムはインタ□ネットにアクセスできないことに注意してください。

- 音声表現: **wav2vec 2.0**・**Whisper** エンコ□ダ□も特□抽出器として使用できます。 [wav2vec モデルカ□ド](#)
- 自動音声認識 (ASR) : **OpenAI Whisper** (任意のサイズ)。 [Whisper モデルカ□ド](#)

- 言語モデル: **Qwen2.5-0.5B** °zero-shot で使用することも、提供された train 分割で **fine-tuning** して使用することもできます。 [Qwen モデルカード](#)

10 分の制限時間には、□□□に行う訓練または **fine-tuning** と、評価セットに□する推論の□□が含まれなければならないことに注意してください。

提出方法

- `solution.ipynb` を開き、すべてのセルを□□してください。作業ディレクトリに `answers.json` が書き□まれ、`dataset/test_public/` 内のすべての□□□100 □□□について置□が含まれていることを確認してください。□□□には、**notebook** が非公開テストセット上で再□行され、そこで生成された `answers.json` が□□されます。
- 必要であれば解答を改善してください。改善しなくてもかまいません。ベ□スラインだけでもパイプラインを□□できます。
- JupyterLab の左サイドバ□にある **Git** タブを開いてください。
- `solution.ipynb` を **Stage** してください（その横にある + アイコン）。
- **commit message** を入力し、**Commit** をクリックしてください。
- 上向き矢印付きの雲アイコンをクリックして **push** してください。
- この **Contest** ペ□ジに□り、**Submit** をクリックしてください。

`solution.ipynb` という名前のファイルをちょうど 1 つ提出してください。